IES AUGUSTÓBRIGA • **IABD**

Programación de **Inteligencia Artificial**

PIA • Tarea 01

Constantino Alexopoulos Real

20 de enero de 2026

Tarea 01

PIA - PROGRAMACIÓN DE INTELIGENCIA ARTIFICIAL

Constantino Alexopoulos Real

Apartado 1: Crear cuenta en GitHub y crear un repositorio.

Accede a la página web de GitHub y sigue los pasos para registrarte y crearte una cuenta. Cuando se te pida que especifiques si eres alumno o profesor, pulsa en la opción que aparece en la parte inferior de la pantalla “skip personalization”. Tras concluir el proceso de registro, crea tu primer repositorio, incluyendo, de momento, un archivo pdf en el que añadas, una breve explicación de cómo lo has hecho.

Apartado 2: Resolver ciertos problemas en Python.

Dado que a lo largo del año vamos a tener que trabajar bastante con Python, es necesario tener cierta base sobre los aspectos básicos del lenguaje. Para ello se propone la realización de los siguientes ejercicios que deberán ser subidos al repositorio GitHub del Apartado 1.

- Problema 1. Procesamiento de una lista de enteros.
  Crea una función que reciba una lista de enteros por parámetro y devuelva otra lista, de acuerdo a las siguientes acciones:
  1. Eliminar los números negativos de la lista.
  2. Eliminar los valores que están repetidos, quedándonos con uno de ellos.
  3. Ordenar los números resultantes de menor a mayor.

  Por ejemplo, si le pasara [4, -1, 2, 4, 3, -5, 2], debería retornar [2,3,4].

- Problema 2. Frecuencia de palabras en un texto.
  Escribe una función que reciba por parámetro una lista de palabras y la ruta a un fichero de texto y devuelva un diccionario que muestre cuantas veces aparecen las distintas palabras de la lista en el fichero de texto. Haz un pequeño programa que la ponga a prueba.
  Requisitos:
  1. Eliminar signos de puntuación y convertir todo a minúsculas.
  2. Usar un diccionario donde la clave sea la palabra y el valor su frecuencia.
  3. Mostrar las palabras y sus frecuencias de forma ordenada por la palabra.

- Problema 3. Trabajo con conjuntos
  Escribe una función que reciba dos listas de enteros y devuelva un diccionario con la siguiente información (ES OBLIGATORIO USAR CONJUNTOS PARA CALCULARLOS):
  1. La intersección de ambos conjuntos (elementos comunes).
  2. La unión de ambos conjuntos (todos los elementos sin duplicados).
  3. La diferencia simétrica (elementos que están en uno u otro conjunto, pero no en ambos).

Apartado 3: Consultar competición en plataforma de IA Kaggle.

Crea una cuenta en Kaggle y haz las siguientes tareas:
o Accede a una competición activa.
o Descarga el dataset usado para esa competición.
o Sube al repositorio Github del primer apartado, un documento con pantallazos de cómo has realizado el proceso y del dataset descargado.

Índice de contenidos

Apartado 1:
Crear cuenta en Github 04

Apartado 2: problemas en Python

Problema 1 06

Problema 2 06

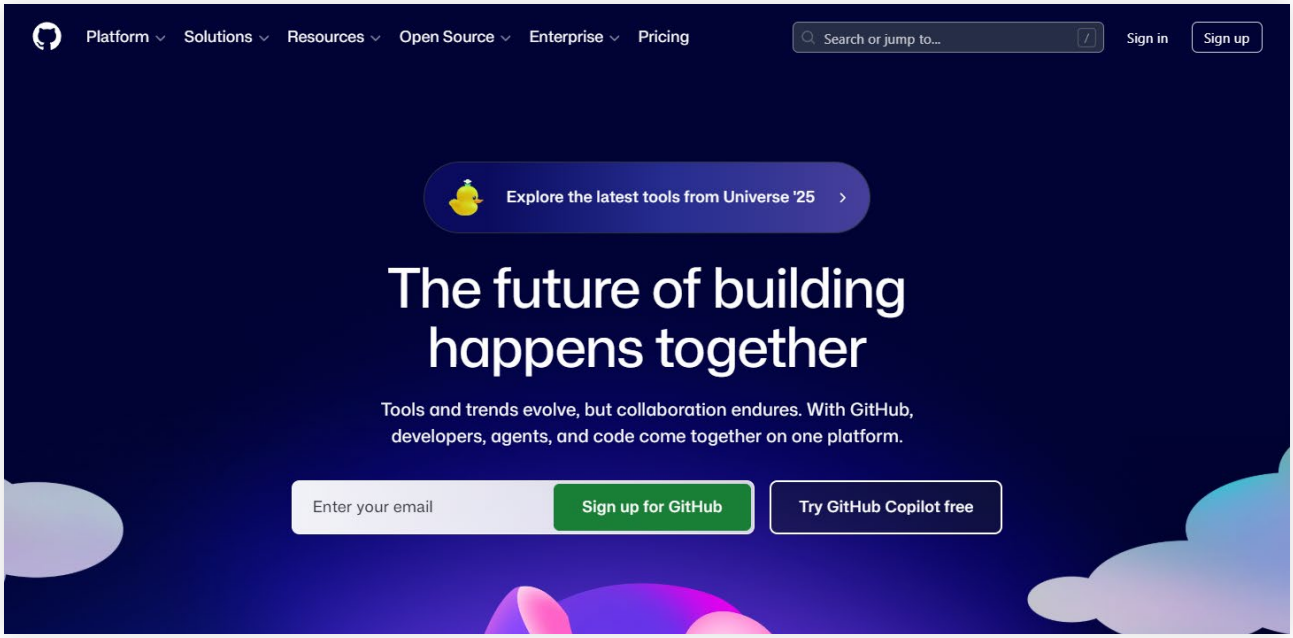
Problema 3 07

Apartado 3:
Consultar competición en *Kaggle* 08

Apartado 1: Crear cuenta en Github

Pasos detallados para configurar una cuenta:

En nuestro navegador vamos al sitio oficial: *github.com*.



Iniciamos el registro haciendo clic en el botón *"Sign up"* o en el banner central.

Introducimos nuestros datos: *GitHub* utiliza una interfaz interactiva tipo "terminal". Tenemos que ingresar:

- Correo electrónico.
- Una contraseña segura.
- Un nombre de usuario: nuestra identidad en la plataforma. En mi caso es *Cantak75*.

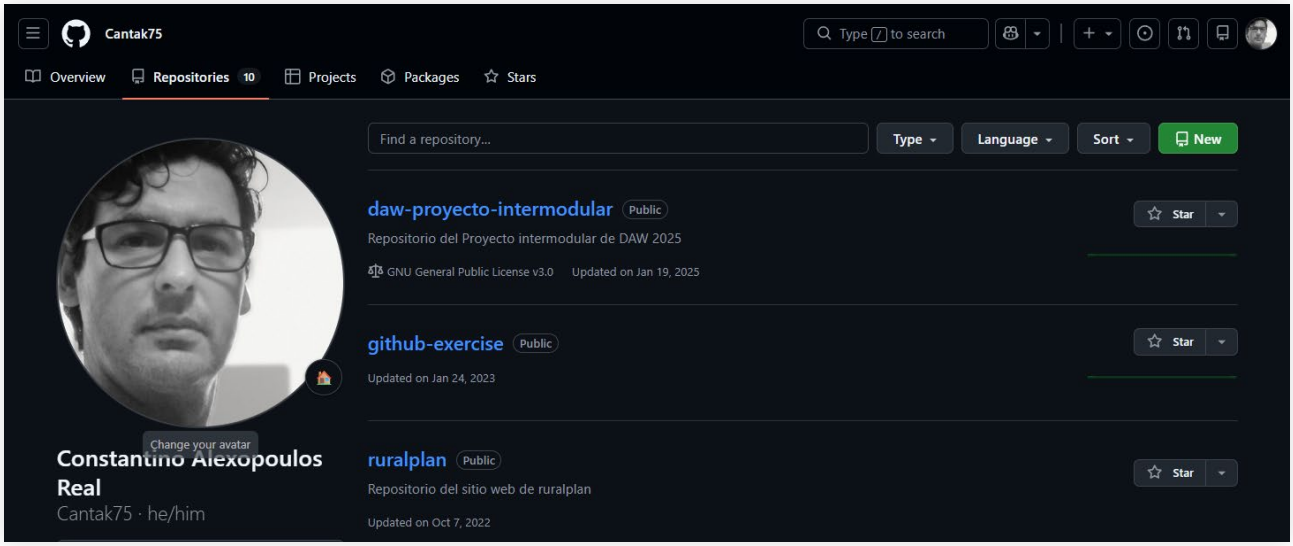
Preferencias de marketing: nos preguntarán si queremos recibir noticias por correo.

Verificación de seguridad: debemos resolver un breve acertijo.

Verificamos nuestro correo: *GitHub* nos enviará un código de 8 dígitos a nuestro email. Lo copiamos y lo pegamos en la página de registro.

Configuración Inicial

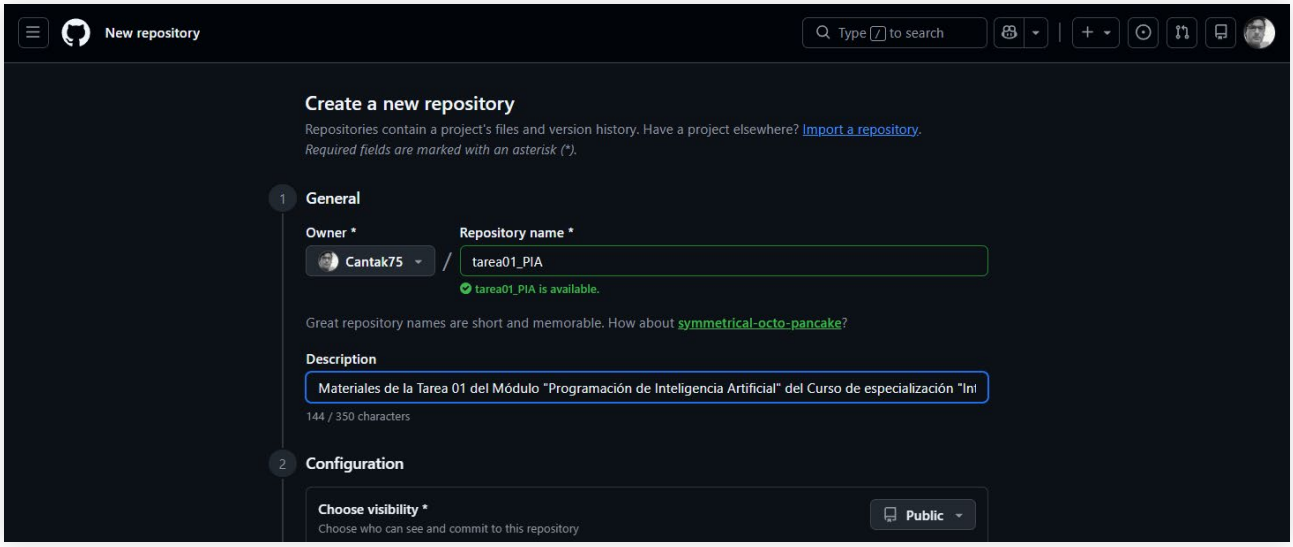
Una vez dentro, tenemos que contestar a unas preguntas opcionales sobre cuántas personas trabajarán con nosotros y si somos estudiantes o profesores. Podemos omitir esto (pulsando *"Skip personalization"* al final).



Creación de repositorios

Una vez tenemos nuestra cuenta, vamos a la sección Repositories y creamos un nuevo repositorio donde guardaremos los archivos de la Tarea 01 de PIA. Para ello hacemos click en el botón NEW y añadimos los datos para el repositorio.

Una vez rellenamos nombre y descripción hacemos click en el botón “Create repository”.



Apartado 2: problemas en Python

• Problema 1. Procesamiento de una lista de enteros.

Crea una función que reciba una lista de enteros por parámetro y devuelva otra lista, de acuerdo a las siguientes acciones:

1. Eliminar los números negativos de la lista.
2. Eliminar los valores que están repetidos, quedándonos con uno de ellos.
3. Ordenar los números resultantes de menor a mayor.

Por ejemplo, si le pasara [4, -1, 2, 4, 3, -5, 2], debería retornar [2,3,4].

```
def procesar_lista(lista):

    # 1. Eliminar números negativos
    positivos = [x for x in lista if x ≥ 0]
    # 2. Eliminar repetidos usando un set
    sin_repetidos = set(positivos)
    # 3. Ordenar de menor a mayor
    resultado = sorted(sin_repetidos)

    return resultado
```

• Problema 2. Frecuencia de palabras en un texto.

Escribe una función que reciba por parámetro una lista de palabras y la ruta a un fichero de texto y devuelva un diccionario que muestre cuantas veces aparecen las distintas palabras de la lista en el fichero de texto. Haz un pequeño programa que la ponga a prueba. Requisitos:

1. Eliminar signos de puntuación y convertir todo a minúsculas.
2. Usar un diccionario donde la clave sea la palabra y el valor su frecuencia.
3. Mostrar las palabras y sus frecuencias de forma ordenada por la palabra.

```
def contar_palabras(lista_palabras, ruta_fichero):
    # Crear diccionario con las palabras de la lista con frecuencia = 0
    frecuencia_palabras = {palabra.lower(): 0 for palabra in lista_palabras}

    with open(ruta_fichero, “r”, encoding=”utf-8”) as fichero:
        for linea in fichero:
            # 1. Eliminar signos de puntuación y pasar a minúsculas
            linea = linea.lower()
            linea = linea.translate(str.maketrans(“”, “”, string.punctuation))

            palabras_linea = linea.split()

            # 2. Contar las palabras de la lista
            for palabra in palabras_linea:
                if palabra in frecuencia_palabras:
                    frecuencia_palabras[palabra] += 1

    return frecuencia_palabras
```

• Problema 3. Trabajo con conjuntos

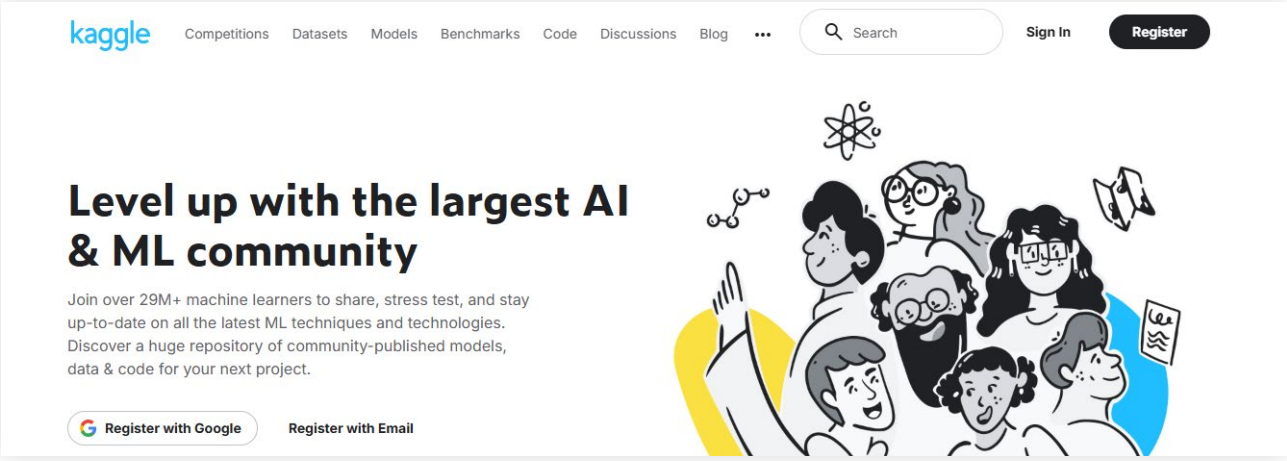
Escribe una función que reciba dos listas de enteros y devuelva un diccionario con la siguiente información (ES OBLIGATORIO USAR CONJUNTOS PARA CALCULARLOS):

1. La intersección de ambos conjuntos (elementos comunes).
2. La unión de ambos conjuntos (todos los elementos sin duplicados).
3. La diferencia simétrica (elementos que están en uno u otro conjunto, pero no en ambos).

```
def operaciones_conjuntos(lista1, lista2):
    # Convertir listas a conjuntos
    conjunto1 = set(lista1)
    conjunto2 = set(lista2)

    resultado = {
        “interseccion”: conjunto1 & conjunto2,
        “union”: conjunto1 | conjunto2,
        “diferencia_simetrica”: conjunto1 ^ conjunto2
    }

    return resultado

lista_a = [1, 2, 3, 4, 5]
lista_b = [1, 4, 5, 5, 6, 7]

resultado = operaciones_conjuntos(lista_a, lista_b)
```

Apartado 3: Consultar competición en plataforma de IA Kaggle

Crea una cuenta en Kaggle y haz las siguientes tareas:
o Accede a una competición activa.
o Descarga el dataset usado para esa competición.
o Sube al repositorio Github del primer apartado, un documento con pantallazos de cómo has realizado el proceso y del dataset descargado.



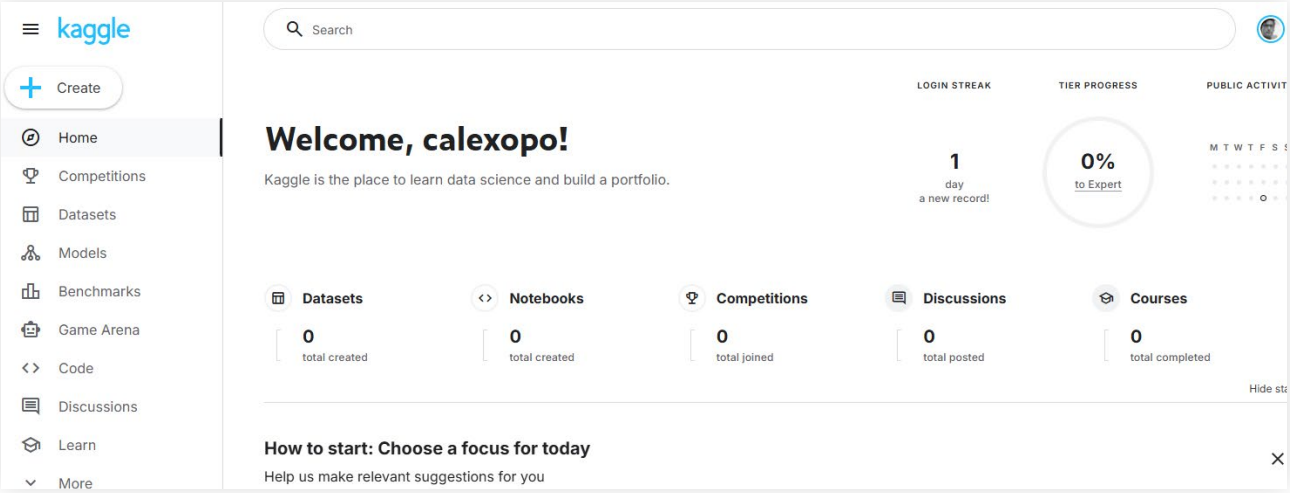
Pasos detallados para configurar una cuenta:

Entramos en https://www.kaggle.com
Hacemos clic en *"Register"* (arriba a la derecha).

Elegimos una opción de registro:

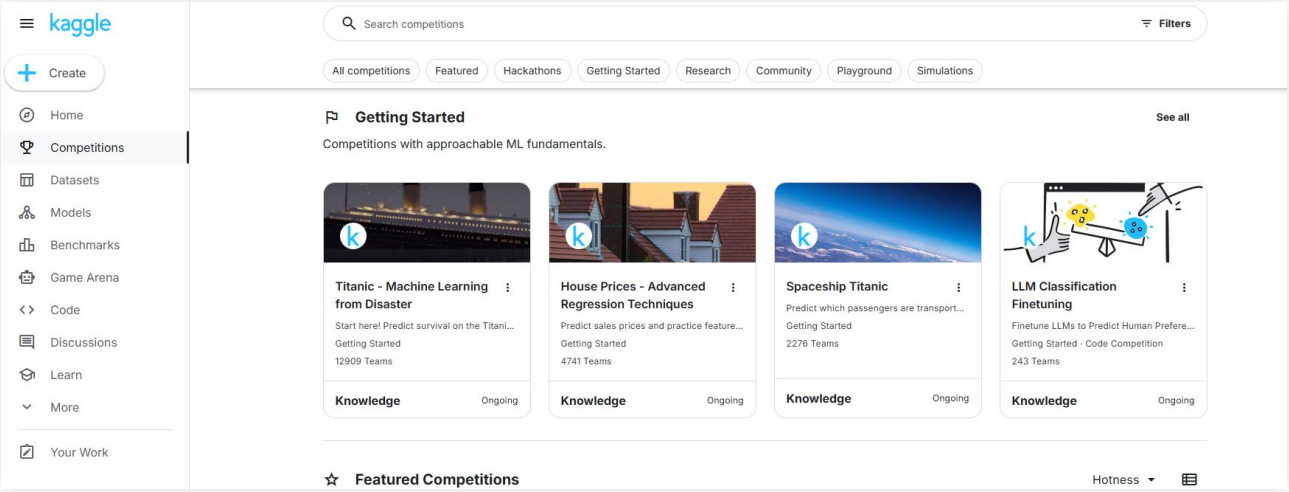
Con cuenta de Google o con email y contraseña.
Completamos los datos solicitados y confirmamos el registro.

Una vez dentro, veremos el mensaje de bienvenida, nuestra foto y estadísticas.

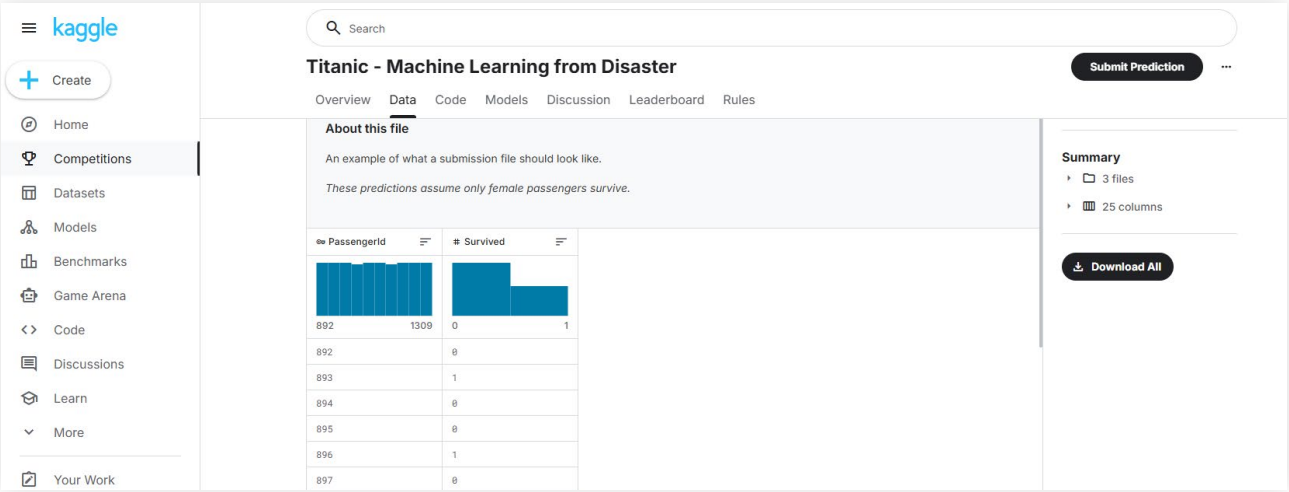


Para acceder a una competición activa, en el menú lateral de *Kaggle*, hacemos clic en "*Competitions*".

Aparecerá una lista de competiciones. Seleccionamos una competición activa.
En mi caso selecciono: *Titanic - Machine Learning from Disaster*). que es una competición que está activa indifinidamente y sirve como primer desafío para familiarizarse con la plataforma de *Kaggle*.



Una vez dentro de la página de la competición leemos la información de la misma y nos descargamos el dataset. Para ello vamos a la pestaña *"Data"*, hacemos scroll hacia abajo y pulsamos en *"Download All"* para descargar todos los archivos del **dataset** (normalmente en formato .csv).



1. Para subir los archivos a nuestro repositorio de Github inicializamos Git en la carpeta desde la que estamos realizando la tarea, vamos a la terminal, nos situamos dentro de ella y ejecutamos:

```
git init
```

Esto crea una carpeta oculta .git donde se guardará todo el historial.

```
Administrador: Símbolo del sistema

D:\10 - FP\03 - IABD\PIA - Programación de Inteligencia Artificial\Tarea 01>git init
Reinitialized existing Git repository in D:/10 - FP/03 - IABD/PIA - Programación de Inteligencia Artificial/Tarea 01/.git/

D:\10 - FP\03 - IABD\PIA - Programación de Inteligencia Artificial\Tarea 01>
```

Para subir nuestros archivos, seguiremos el siguiente ciclo de tres pasos:

Añadir > Confirmar > Empujar.

Para añadir todos los archivos de la carpeta al *"Staging Area"* ejecutamos:

```
git add .
```

```
Administrador: Símbolo del sistema

D:\10 - FP\03 - IABD\PIA - Programación de Inteligencia Artificial\Tarea 01>git init
Reinitialized existing Git repository in D:/10 - FP/03 - IABD/PIA - Programación de Inteligencia Artificial/Tarea 01/.git/

D:\10 - FP\03 - IABD\PIA - Programación de Inteligencia Artificial\Tarea 01>git add .

D:\10 - FP\03 - IABD\PIA - Programación de Inteligencia Artificial\Tarea 01>
```

Guardamos los cambios localmente (hacemos el Commit, la foto del momento) con un mensaje descriptivo:

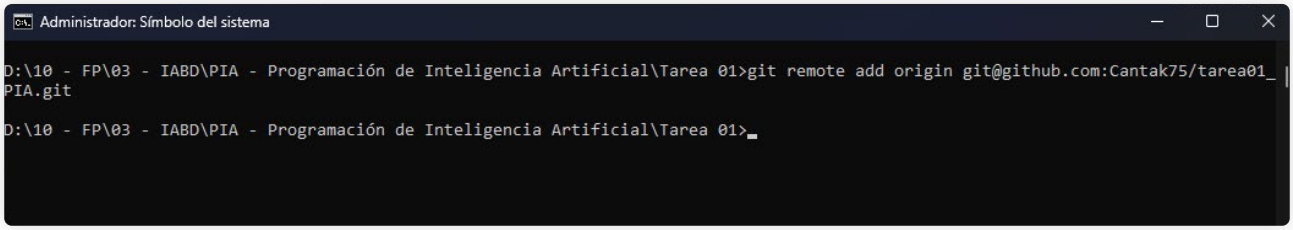
```
Administrador: Símbolo del sistema

D:\10 - FP\03 - IABD\PIA - Programación de Inteligencia Artificial\Tarea 01>git commit -m "Mi primer commit: subiendo archivos de
la Tarea 01 de PIA"
[master (root-commit) 4ef231a] Mi primer commit: subiendo archivos de la Tarea 01 de PIA
 9 files changed, 1829 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 Tarea de la unidad 1 de PIA.pdf
 create mode 100644 ejemplo.txt
 create mode 100644 problema01.py
 create mode 100644 problema02.py
 create mode 100644 problema03.py
 create mode 100644 titanic/gender_submission.csv
 create mode 100644 titanic/test.csv
 create mode 100644 titanic/train.csv

D:\10 - FP\03 - IABD\PIA - Programación de Inteligencia Artificial\Tarea 01>
```

```
git commit –m "Mi primer commit: subiendo archivos de la Tarea 01 de PIA"
```

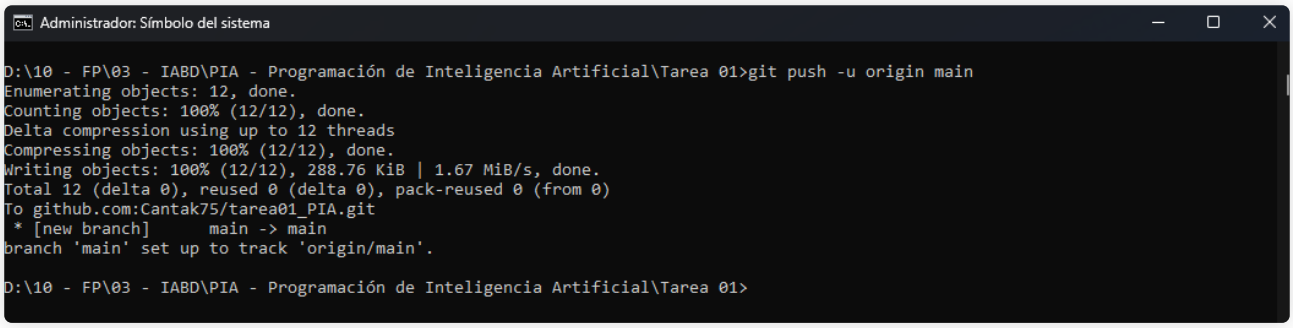
Conectamos con el repositorio remoto (Solo la primera vez).

Necesitamos decirle a Git a dónde vamos a enviar nuestros archivos:

```
git remote add origin git@github.com:Cantak75/tarea01_PIA.git
```



Para subir los archivos (*Push*) a la nube:

```
git push -u origin main
```

Antes de hacer el git add ., creamos el archivo .gitignore.
En este archivo añadimos el nombre de archivos y carpetas que no queremos subir (como node_modules, archivos de configuración personal o carpetas temporales). Git las ignorará automáticamente.